BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH




# BÁO CÁO TỔNG KẾT
# GIÁM SÁT TOÀN DIỆN BẰNG CLOUDWATCH: METRICS, LOGS, ALARMS, DASHBOARD

**MÔN: ĐIỆN TOÁN ĐÁM MÂY**

**HỌC KỲ 1 - NĂM HỌC 2026-2027**

**GIẢNG VIÊN HƯỚNG DẪN: TS. HUỲNH XUÂN PHỤNG**

**THỰC HIỆN:**


1. **Nguyễn Tuấn Khang** **24110025**
2. **Đào Gia Khiêm** **24110026**

# MỤC LỤC


**1. Giới thiệu ..... 5**

**1.1. Vấn đề đặt ra ..... 5**

**1.2. Mục tiêu dự án** ..... 6

**1.3. Phạm vi dự án** ..... 6

**2. Kiến trúc hệ thống ..... 7**

**2.1. Tổng quan kiến trúc ..... 7**

**2.2. Các dịch vụ AWS sử dụng và lý do lựa chọn ..... 8**

**2.3. Luồng dữ liệu của hệ thống ..... 9**

**2.4. Các ràng buộc của AWS Academy Learner Lab và cách xử lý 10**

**3. Tích hợp tài nguyên ..... 10**

**3.1. VPC, EC2, Log group, Cloudwatch Agent ..... 10**

3.1.1 Xây dựng mạng riêng ảo VPC ..... 10

3.1.2 Triển khai máy chủ Amazon EC2 ..... 11

3.1.3 Cấu hình CloudWatch Log Groups ..... 12

3.1.4 Cài đặt và cấu hình CloudWatch Agent ..... 13

**3.2. Alarm, SNS, Dashboard, Metric Filter ..... 16**

**3.2.1 Tạo Metric Filter đếm lỗi HTTP 5xx** ..... 16

**3.2.2 Tạo SNS Topic và đăng ký email nhận cảnh báo** ..... 17

**3.2.3 Tạo CloudWatch Alarm cho lỗi HTTP 5xx** ..... 18

**3.2.4 Xây dựng CloudWatch Dashboard** ..... 19

**4. Bảo mật ..... 20**

**5. Kiểm thử ..... 22**

**5.1. Mục tiêu và môi trường kiểm thử ..... 22**

**5.2. Các kịch bản kiểm thử ..... 22**

**5.3. Kết quả kiểm thử chi tiết ..... 23**

5.3.1 Kiểm thử lỗi HTTP 5xx ..... 23

**5.3.2 Kiểm thử lỗi HTTP 404** ..... 25

**5.3.3 Kiểm thử RAM sử dụng cao** ..... 26

**5.3.4 Kiểm thử ổ đĩa đầy** ..... 27
**5.3.5 Kiểm thử CPU sử dụng cao** ..... 28
**5.4. Đánh giá kết quả** ..... **29**
**6. Chi phí** ..... **30**
**7. Vấn đề gặp phải và bài học rút ra** ..... **32**
**8. Phân công và khai báo sử dụng AI** ..... **32**

| Field | Fill in |
|---|---|
| **Course / Class** | Điện toán đám mây |
| **Project No. & Title** | Giám sát toàn diện bằng CloudWatch: metrics, logs, alarms, dashboard (Nhóm: Monitoring & Operations) |
| **Student 1 (Name – ID)** | Nguyễn Tuấn Khang - 24110025 |
| **Student 2 (Name – ID)** | Đào Gia Khiêm – 24110026 |
| **Instructor** | TS. Huỳnh Xuân Phụng |
| **Start date – Due date** | 5/10/2026 – 2/11/2026 |

# 1. Giới thiệu

## 1.1. Vấn đề đặt ra

Trong quá trình vận hành các ứng dụng và dịch vụ trên nền tảng điện toán đám mây, việc theo dõi tình trạng hoạt động của máy chủ và phát hiện kịp thời các sự cố là một yêu cầu quan trọng. Đặc biệt, đối với các máy chủ EC2 chạy ứng dụng web, những vấn đề như mức sử dụng CPU hoặc RAM tăng cao, dung lượng ổ đĩa gần đầy, lỗi máy chủ HTTP và các sự cố phát sinh trong quá trình xử lý yêu cầu có thể ảnh hưởng trực tiếp đến hiệu suất và khả năng cung cấp dịch vụ. Nếu không có một hệ thống giám sát phù hợp, người quản trị sẽ gặp khó khăn trong việc theo dõi tình trạng tài nguyên, xác định nguyên nhân sự cố và đưa ra biện pháp xử lý kịp thời. Bên cạnh đó, việc kiểm tra thủ công thường xuyên cũng tốn thời gian, thiếu tính liên tục và có thể khiến các sự cố không được phát hiện ngay khi xảy ra.

Xuất phát từ những vấn đề trên, nhóm lựa chọn xây dựng hệ thống giám sát máy chủ bằng Amazon CloudWatch trên nền tảng AWS. Hệ thống hướng đến việc tập trung dữ liệu giám sát, thu thập nhật ký hoạt động, phân tích các chỉ số tài nguyên và tự động gửi cảnh báo khi phát hiện những dấu hiệu bất thường. Qua đó, người quản trị có thể nắm bắt tình trạng hệ thống một cách trực quan, giảm sự phụ thuộc vào việc kiểm tra thủ công và nâng cao khả năng phát hiện, xử lý sự.

Đảng và Nhà nước ta trong những năm gần đây đã có nhiều chuyển biến mạnh mẽ về tư duy, đặc biệt là việc đặt văn hóa ngang hàng với kinh tế, chính trị và xã hội. Tuy nhiên, khoảng cách từ nhận thức lý luận đến thực tiễn thi hành vẫn còn nhiều điểm nghẽn. Việc đầu tư cho văn hóa và con người ở nhiều nơi vẫn còn mang tính hình thức, chưa thực sự giải quyết được các bài toán thực tế như phát triển công nghiệp văn hóa, bảo vệ chủ quyền văn hóa số hay xây dựng chuẩn mực con người Việt Nam hiện đại.

Xuất phát từ tầm quan trọng đặc biệt đó, việc thực hiện đề tài “Quan điểm

Hồ Chí Minh về văn hóa và con người. Làm sáng tỏ câu nói 'Văn hóa còn thì dân tộc còn, văn hóa mất thì dân tộc mất' và sự vận dụng của Đảng ta hiện nay" là hết sức cần thiết. Đề tài hướng tới việc tái thiết lập góc nhìn đúng đắn về di sản tư tưởng Hồ Chí Minh, làm rõ giá trị thời đại của câu nói trên, đồng thời đề xuất những giải pháp thực chất giúp cụ thể hóa đường lối của Đảng vào đời sống xã hội hiện nay.

### 1.2. Mục tiêu dự án

Mục tiêu chính của dự án là xây dựng một hệ thống giám sát tập trung cho máy chủ Amazon EC2 sử dụng các dịch vụ của AWS, trong đó Amazon CloudWatch đóng vai trò trung tâm trong việc thu thập, lưu trữ, phân tích và hiển thị dữ liệu giám sát. Cụ thể, nhóm triển khai CloudWatch Agent trên máy chủ EC2 để thu thập các chỉ số quan trọng như CPU, RAM và Disk, đồng thời ghi nhận nhật ký truy cập và nhật ký lỗi của máy chủ web Nginx.

Bên cạnh việc thu thập dữ liệu, dự án còn hướng đến khả năng phát hiện và cảnh báo tự động khi hệ thống xuất hiện lỗi. Nhóm sử dụng CloudWatch Metric Filter để lọc và thống kê các lỗi HTTP 5xx từ Access Logs, sau đó thiết lập CloudWatch Alarm để theo dõi số lượng lỗi và kích hoạt cảnh báo khi vượt ngưỡng đã xác định. Amazon SNS được tích hợp nhằm gửi thông báo đến người quản trị, giúp phát hiện sự cố kịp thời. Ngoài ra, nhóm xây dựng CloudWatch Dashboard để tổng hợp các chỉ số CPU, RAM, Disk và HTTP 5xx trên cùng một giao diện, hỗ trợ việc quan sát và đánh giá tình trạng hệ thống.

### 1.3. Phạm vi dự án

Trong phạm vi dự án, nhóm tập trung xây dựng và thử nghiệm hệ thống giám sát cho một máy chủ Amazon EC2 sử dụng hệ điều hành Amazon Linux và máy chủ web Nginx. Hạ tầng triển khai bao gồm các thành phần cơ bản như Amazon VPC, IAM Role, Amazon S3 và EC2. Các tài nguyên được cấu hình nhằm đáp ứng yêu cầu kết nối, vận hành và quản lý hệ thống trên AWS.

Phạm vi giám sát tập trung vào ba nhóm dữ liệu chính. Thứ nhất là các chỉ số tài nguyên máy chủ, bao gồm CPU, RAM và Disk. Thứ hai là nhật ký hoạt động của Nginx, bao gồm Access Logs và Error Logs, được thu thập và gửi đến Amazon CloudWatch Logs thông qua CloudWatch Agent. Thứ ba là các lỗi HTTP 5xx được phát hiện thông qua Metric Filter, từ đó cung cấp dữ liệu cho CloudWatch Alarm thực hiện giám sát và cảnh báo. Amazon SNS được sử dụng để gửi thông báo khi Alarm chuyển sang trạng thái cảnh báo, trong khi CloudWatch Dashboard cung cấp giao diện trực quan để theo dõi tình trạng hoạt động của máy chủ.

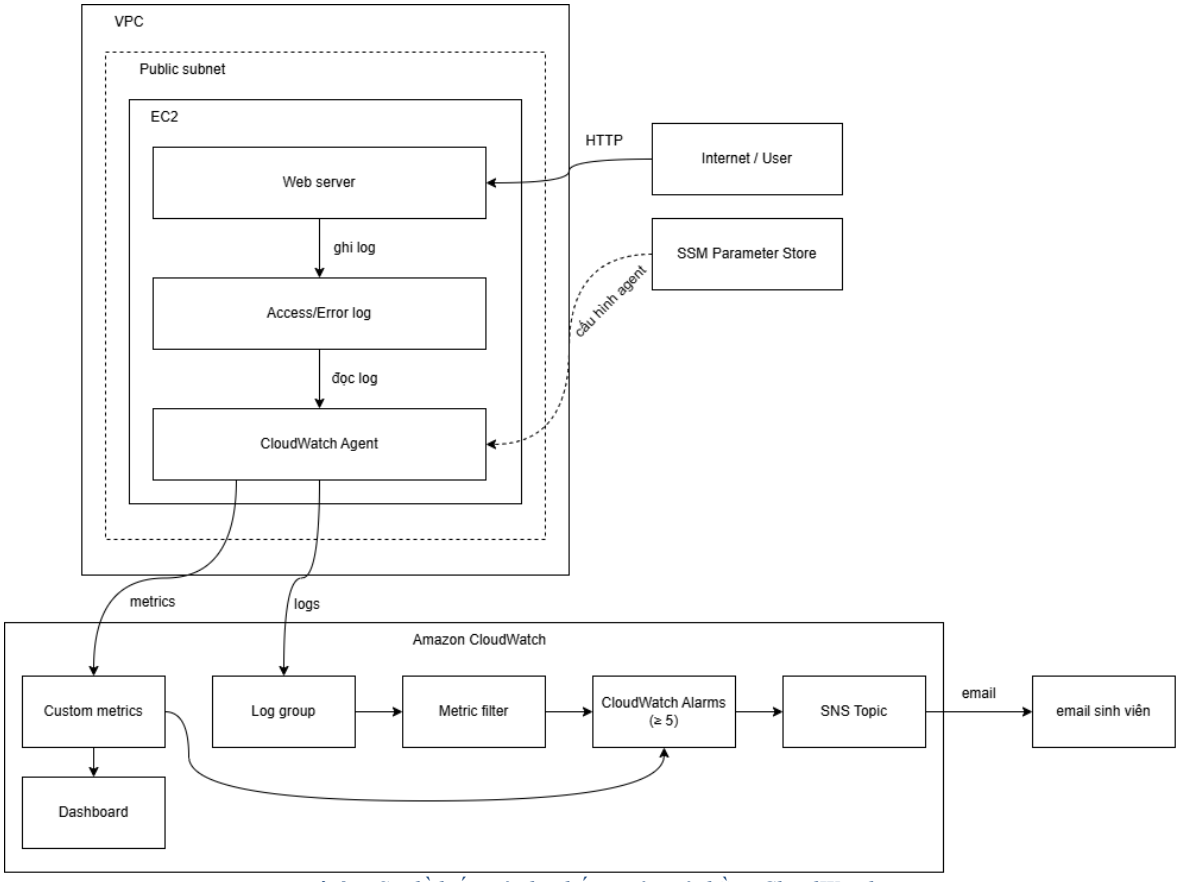
Ngoài ra, dự án còn giới hạn trong việc kiểm tra các chức năng giám sát, mô phỏng lỗi HTTP 500 để đánh giá khả năng phát hiện sự cố, kiểm tra hoạt động của hệ thống cảnh báo và xác nhận dữ liệu được hiển thị trên Dashboard. Nhóm cũng thực hiện rà soát quyền truy cập, đánh giá chi phí sử dụng dịch vụ AWS, tối ưu thời gian lưu trữ Logs và dọn dẹp các tài nguyên không còn cần thiết. Dự án chủ yếu phục vụ mục đích học tập, nghiên cứu và thử nghiệm trên môi trường AWS, chưa hướng đến triển khai hệ thống giám sát quy mô lớn hoặc môi trường sản xuất có yêu cầu cao về tính sẵn sàng, khả năng mở rộng và tính liên tục của dịch vụ.

## 2. Kiến trúc hệ thống

### 2.1. Tổng quan kiến trúc

Hệ thống giám sát được thiết kế xoay quanh một máy chủ Amazon EC2 chạy Nginx đặt trong mạng riêng ảo Amazon VPC. Trên máy chủ này, CloudWatch Agent đóng vai trò thu thập dữ liệu: các chỉ số RAM, Disk và hai file nhật ký của Nginx (access.log, error.log) được gửi về Amazon CloudWatch. Tại CloudWatch, dữ liệu được lưu trữ, biến đổi và hiển thị: Log Groups lưu nhật ký, Metric Filter chuyển các dòng log lỗi HTTP 5xx thành chỉ số, Alarm theo dõi các chỉ số theo ngưỡng, còn Dashboard tổng hợp toàn bộ thông tin trên một giao diện duy nhất. Khi Alarm chuyển sang trạng thái cảnh báo, Amazon SNS gửi email thông báo đến người quản trị.

Bên cạnh luồng giám sát chính, AWS Systems Manager Parameter Store được dùng để lưu cấu hình của CloudWatch Agent, Amazon S3 lưu tài liệu và template của dự án, còn IAM Role cấp quyền để EC2 làm việc với các dịch vụ trên mà không cần lưu khóa truy cập trên máy chủ. Sơ đồ tổng thể của hệ thống được thể hiện ở hình dưới đây.




*Hình 1 Sơ đồ kiến trúc hệ thống giám sát bằng CloudWatch*

## 2.2. Các dịch vụ AWS sử dụng và lý do lựa chọn

Bảng dưới đây tóm tắt vai trò của từng dịch vụ trong hệ thống và lý do nhóm lựa chọn dịch vụ đó.

| Dịch vụ | Vai trò trong hệ thống | Lý do lựa chọn |
|---|---|---|
| Amazon VPC | Mạng riêng 10.0.0.0/16 gồm subnet, route table và Internet Gateway cho EC2 | Tạo môi trường mạng riêng, kiểm soát được kết nối vào/ra của máy chủ |
| Amazon EC2 | Máy chủ Amazon Linux chạy web server Nginx, là đối tượng được giám sát | Cung cấp tài nguyên (CPU, RAM, Disk) và nhật ký thực tế để giám sát |

| Dịch vụ | Vai trò trong hệ thống | Lý do lựa chọn |
|---|---|---|
| CloudWatch Agent | Thu thập RAM, Disk và log Nginx trên EC2 rồi gửi lên CloudWatch | EC2 mặc định không cung cấp chỉ số RAM và Disk, cần Agent để bổ sung |
| CloudWatch Metrics | Lưu trữ các chỉ số CPU, RAM, Disk và số lỗi HTTP 5xx | Tích hợp sẵn với EC2 và Agent, hỗ trợ vẽ biểu đồ và đặt Alarm |
| CloudWatch Logs | Lưu Access Logs và Error Logs của Nginx (giữ 7 ngày) | Tập trung nhật ký, dễ tìm kiếm; giới hạn thời gian lưu để tiết kiệm chi phí |
| Metric Filter | Đếm các dòng log có mã trạng thái 5xx và tạo thành chỉ số | Cho phép đặt cảnh báo dựa trên nội dung log, không cần viết thêm mã |
| CloudWatch Alarm | Theo dõi chỉ số, so sánh với ngưỡng và kích hoạt hành động | Phát hiện sự cố tự động, không phải kiểm tra thủ công |
| Amazon SNS | Gửi email cảnh báo khi Alarm chuyển sang ALARM | Tích hợp trực tiếp với Alarm, dễ cấu hình, không cần tự dựng máy chủ mail |
| CloudWatch Dashboard | Hiển thị CPU, RAM, Disk và lỗi HTTP 5xx trên một giao diện | Giúp quan sát tình trạng hệ thống một cách trực quan |
| Parameter Store | Lưu cấu hình CloudWatch Agent (AmazonCloudWatch-Linux-EC2-Config) | Quản lý cấu hình tập trung, dễ tái sử dụng khi triển khai lại Agent |
| Amazon S3 | Lưu tài liệu và template của dự án | Lưu trữ bền vững, chi phí thấp |
| IAM Role (LabRole) | Cấp quyền cho EC2 gửi dữ liệu lên CloudWatch và đọc Parameter Store | Learner Lab cung cấp sẵn LabRole và không cho tạo IAM Role mới |

### 2.3. Luồng dữ liệu của hệ thống

Dữ liệu giám sát đi qua hệ thống theo các bước sau:

**Bước 1:** Người dùng hoặc script kiểm thử gửi yêu cầu HTTP đến Nginx trên EC2 qua Internet Gateway.

**Bước 2:** Nginx xử lý yêu cầu và ghi lại vào /var/log/nginx/access.log (yêu cầu và mã trạng thái) và /var/log/nginx/error.log (lỗi).

**Bước 3:** CloudWatch Agent, dùng cấu hình lưu trong Parameter Store và quyền của LabRole, mỗi 60 giây thu thập mem_used_percent và disk_used_percent, đồng thời đọc hai file log rồi gửi lên CloudWatch (namespace CWAgent và hai Log Groups tương ứng).

**Bước 4:** Metric Filter trên Log Group access đếm các dòng có mã trạng thái

5xx và ghi thành một chỉ số riêng trong CloudWatch Metrics.

**Bước 5:** CloudWatch Alarm đánh giá liên tục chỉ số lỗi 5xx cùng các chỉ số CPU, RAM, Disk so với ngưỡng đã đặt.

**Bước 6:** Khi vượt ngưỡng, Alarm chuyển sang trạng thái ALARM và gửi thông điệp đến SNS Topic; SNS chuyển thông điệp đến email đã đăng ký.

**Bước 7:** CloudWatch Dashboard đọc các chỉ số trên và hiển thị cho người quản trị theo dõi.

### 2.4. Các ràng buộc của AWS Academy Learner Lab và cách xử lý

Do triển khai trên môi trường AWS Academy Learner Lab, nhóm phải thiết kế kiến trúc phù hợp với một số giới hạn của môi trường này:

- **Không tạo được IAM User/Role mới:** nhóm sử dụng LabRole và LabInstanceProfile có sẵn để cấp quyền cho EC2 và các dịch vụ liên quan.
- **Giới hạn Region:** môi trường chỉ cho phép dùng Region us-east-1 hoặc us-west-2. Nhóm kiểm tra Region từ tuần 1 và đặt toàn bộ tài nguyên trong us-east-1.
- **Giới hạn tài nguyên:** EC2 chỉ được chọn loại từ nano đến large, ổ EBS tối đa 100 GB. Nhóm dùng một máy chủ t3.micro, đủ cho nhu cầu giám sát và kiểm thử.
- **Quy định về quản lý tài nguyên:** mọi tài nguyên phải gắn tag Project và Owner, thời gian lưu log là 7 ngày, và phải dừng hoặc xóa tài nguyên sau mỗi buổi làm việc để tránh phát sinh chi phí.
- **Phiên làm việc có thời gian giới hạn:** tài nguyên có thể bị dừng khi phiên kết thúc, vì vậy nhóm lưu cấu hình Agent trong Parameter Store và ghi lại các lệnh thực hiện để dựng lại nhanh khi cần.

## 3. Tích hợp tài nguyên

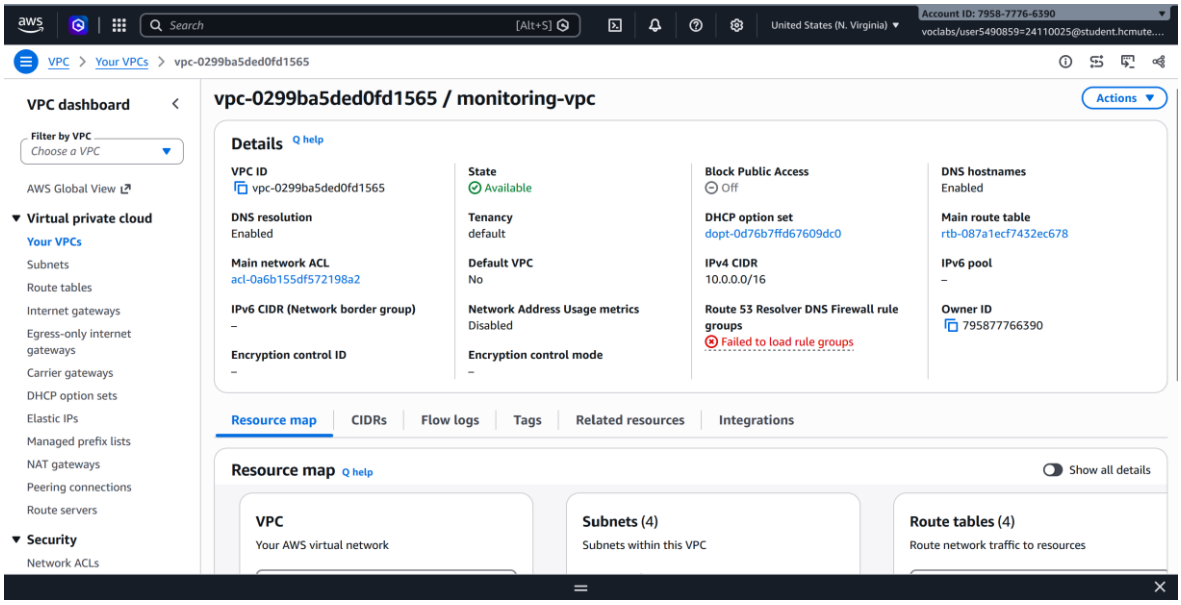
### 3.1. VPC, EC2, Log group, Cloudwatch Agent

3.1.1 Xây dựng mạng riêng ảo VPC

Đầu tiên, nhóm tiến hành xây dựng hạ tầng mạng trên nền tảng Amazon Web Services bằng dịch vụ Amazon Virtual Private Cloud. VPC được tạo

nhằm cung cấp một môi trường mạng riêng cho các tài nguyên của dự án, đặc biệt là máy chủ Amazon EC2. Nhóm cấu hình VPC với dải địa chỉ IPv4 CIDR là 10.0.0.0/16, cho phép hệ thống sử dụng một không gian địa chỉ IP riêng để triển khai các thành phần mạng.

Sau khi tạo VPC, nhóm thiết lập thuộc tính DNS Hostnames để hỗ trợ việc phân giải tên miền cho các tài nguyên trong mạng. Đồng thời, các thẻ Tag được gắn vào VPC nhằm thuận tiện cho việc nhận diện và quản lý tài nguyên. Cụ thể, nhóm sử dụng Tag Project=CloudWatch-Monitoring để xác định tài nguyên thuộc dự án và Owner=TuanKhang để nhận diện người phụ trách.




*Hình 2 Ảnh VPC trong AWS Console*

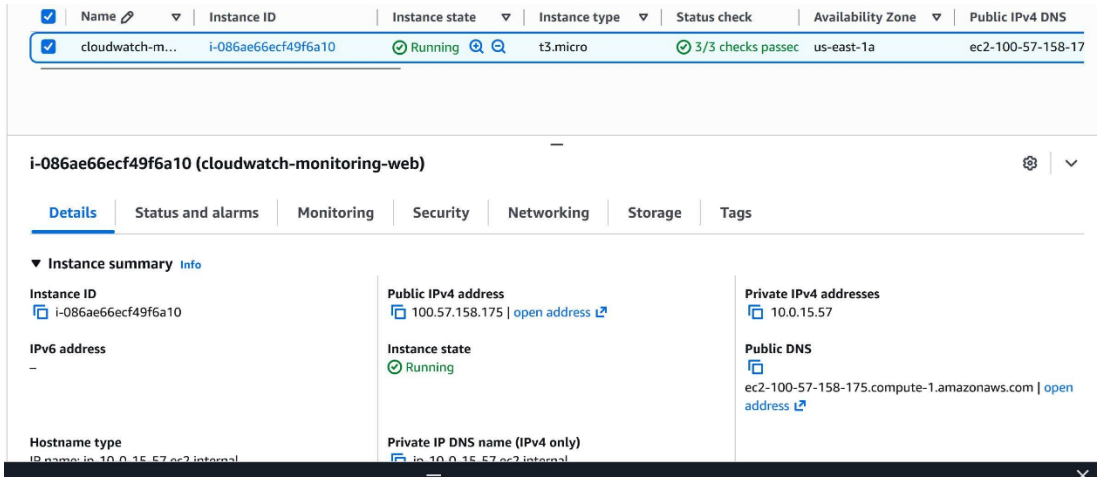
### 3.1.2 Triển khai máy chủ Amazon EC2

Sau khi hoàn thành việc xây dựng hạ tầng mạng, nhóm tiến hành khởi tạo một máy chủ ảo bằng dịch vụ Amazon Elastic Compute Cloud. Máy chủ sử dụng hệ điều hành Amazon Linux, đóng vai trò là môi trường chạy ứng dụng web thử nghiệm và cung cấp dữ liệu phục vụ quá trình giám sát.

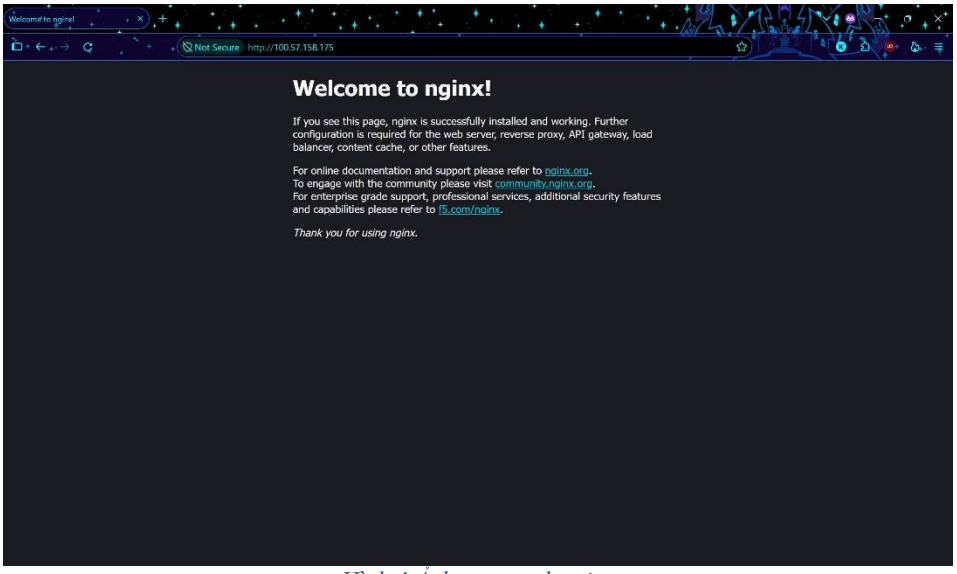
Trong quá trình khởi tạo EC2, nhóm lựa chọn VPC đã tạo trước đó và cấu hình các thông số mạng phù hợp. Security Group được thiết lập để kiểm soát lưu lượng truy cập đến máy chủ. Cổng HTTP 80 được mở để cho phép truy cập ứng dụng web từ trình duyệt theo nhu cầu kiểm thử.

Sau khi khởi tạo thành công, nhóm kết nối đến EC2 thông qua SSH và tiến

hành cài đặt máy chủ web Nginx. Nginx được lựa chọn để triển khai một ứng dụng web thử nghiệm, đồng thời tạo ra các nhật ký truy cập và nhật ký lỗi phục vụ cho quá trình giám sát bằng CloudWatch.




*Hình 3 Ảnh EC2*




*Hình 4 Ảnh trang web nginx*

### 3.1.3 Cấu hình CloudWatch Log Groups

Để theo dõi hoạt động của máy chủ web, nhóm sử dụng Amazon CloudWatch Logs nhằm thu thập và lưu trữ các nhật ký phát sinh trong quá trình Nginx xử lý yêu cầu. Hai loại nhật ký chính được sử dụng là Access Logs và Error Logs.

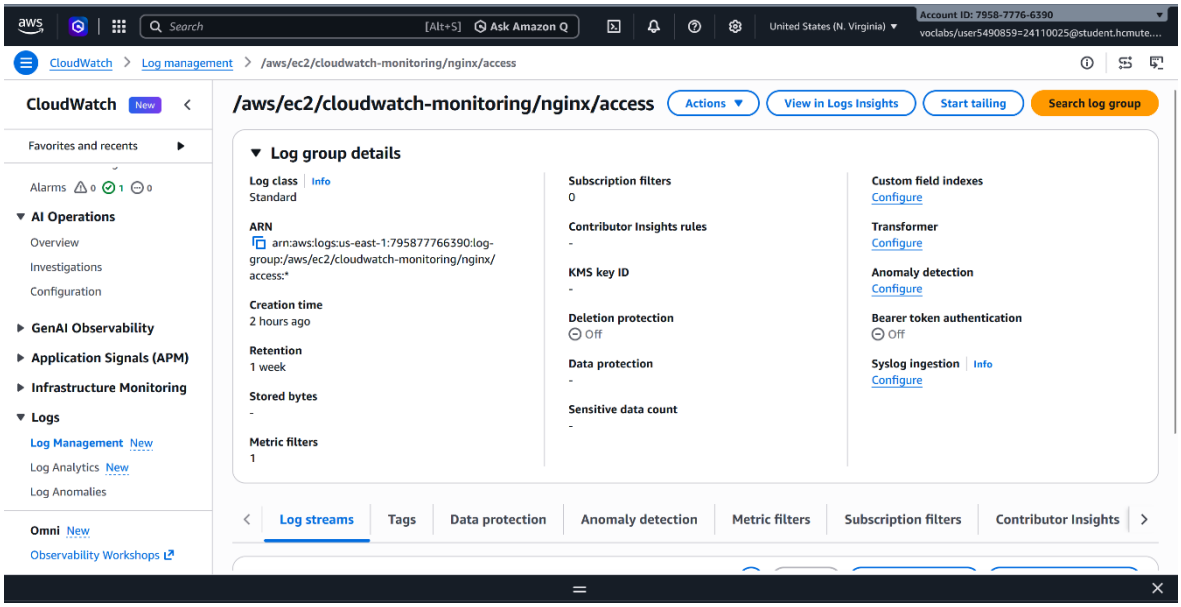
Access Logs ghi nhận thông tin về các yêu cầu được gửi đến máy chủ, bao

gồm địa chỉ IP của người truy cập, thời gian truy cập, đường dẫn yêu cầu và mã trạng thái HTTP trả về. Trong khi đó, Error Logs lưu lại các thông tin liên quan đến lỗi phát sinh trong quá trình vận hành Nginx.

Nhóm tổ chức dữ liệu thành hai Log Groups riêng biệt trên CloudWatch:

- /aws/ec2/cloudwatch-monitoring/nginx/access: lưu trữ nhật ký truy cập của Nginx.
- /aws/ec2/cloudwatch-monitoring/nginx/error: lưu trữ nhật ký lỗi của Nginx.

Việc phân chia Log Groups giúp dữ liệu được tổ chức rõ ràng, thuận tiện cho việc tìm kiếm, phân tích và phát hiện sự cố. Ngoài ra, nhóm thiết lập thời gian lưu trữ Logs là 7 ngày nhằm giới hạn lượng dữ liệu được lưu trữ, hỗ trợ tối ưu chi phí sử dụng dịch vụ CloudWatch.




*Hình 5 Ảnh Log Group access*

### 3.1.4 Cài đặt và cấu hình CloudWatch Agent

Sau khi hoàn thành việc triển khai máy chủ EC2 và tạo các Log Groups, nhóm tiến hành cài đặt Amazon CloudWatch Agent trên máy chủ. CloudWatch Agent là thành phần có nhiệm vụ thu thập các chỉ số tài nguyên hệ thống và nhật ký hoạt động, sau đó gửi dữ liệu về Amazon CloudWatch để phục vụ quá trình giám sát tập trung.

Mặc định, EC2 cung cấp một số chỉ số cơ bản như CPUUtilization,

NetworkIn và NetworkOut. Tuy nhiên, các chỉ số như mức sử dụng RAM và dung lượng ổ đĩa cần được thu thập bổ sung thông qua CloudWatch Agent. Vì vậy, nhóm sử dụng Agent để mở rộng phạm vi giám sát tài nguyên máy chủ.

Nhóm thực hiện cài đặt CloudWatch Agent trên Amazon Linux bằng lệnh:

sudo dnf install amazon-cloudwatch-agent -y

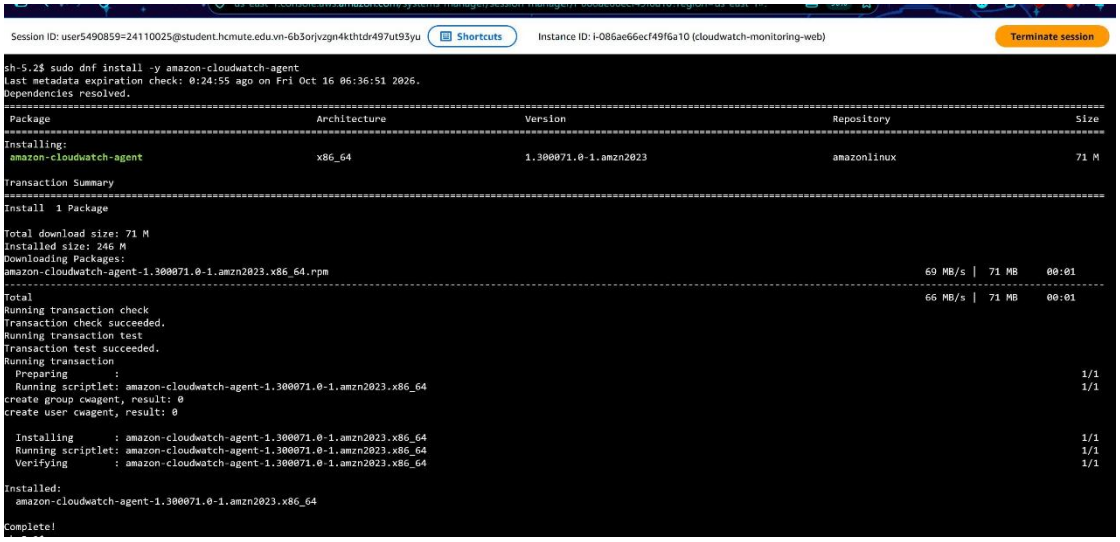
Sau khi cài đặt, nhóm tạo file cấu hình tại đường dẫn: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json

File cấu hình được thiết lập với chu kỳ thu thập dữ liệu là 60 giây. Các chỉ số được thu thập bao gồm mem_used_percent để theo dõi tỷ lệ RAM đã sử dụng và disk_used_percent để theo dõi tỷ lệ dung lượng ổ đĩa đã sử dụng tại phân vùng gốc /. Dữ liệu Metrics được gửi lên CloudWatch với namespace CWAgent và gắn kèm thông tin InstanceId để phân biệt máy chủ gửi dữ liệu.

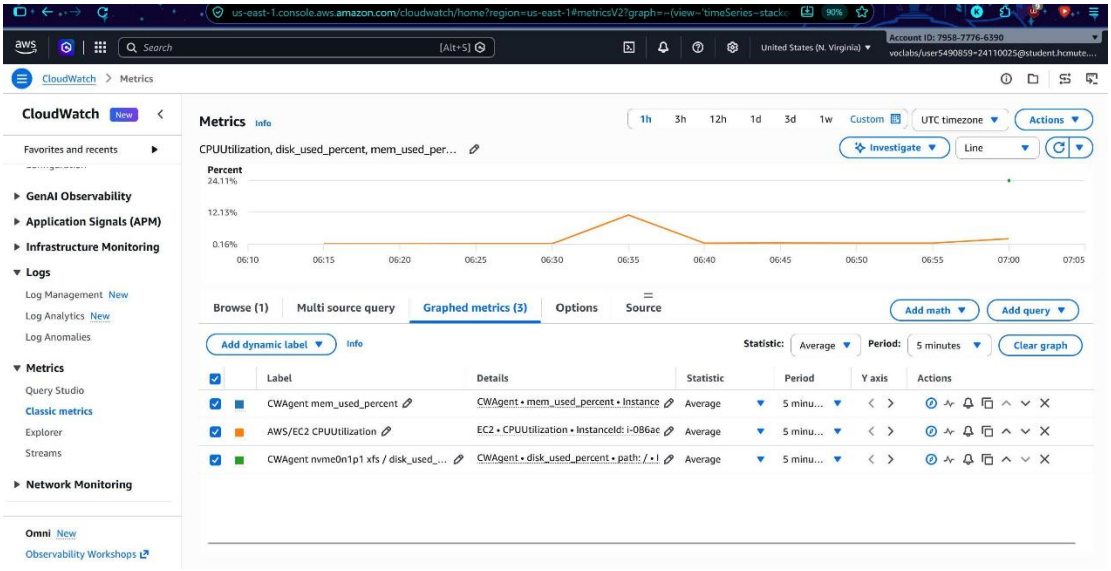
Bên cạnh việc thu thập Metrics, nhóm cấu hình CloudWatch Agent để đọc hai file nhật ký của Nginx là /var/log/nginx/access.log và /var/log/nginx/error.log. Dữ liệu sau đó được chuyển đến hai Log Groups tương ứng đã tạo trên CloudWatch.

Ngoài ra, nhóm sử dụng AWS Systems Manager Parameter Store để lưu trữ file cấu hình CloudWatch Agent với tên tham số AmazonCloudWatch-Linux-EC2-Config. Việc lưu cấu hình tập trung giúp thuận tiện cho việc quản lý và tái sử dụng cấu hình khi cần triển khai lại Agent.

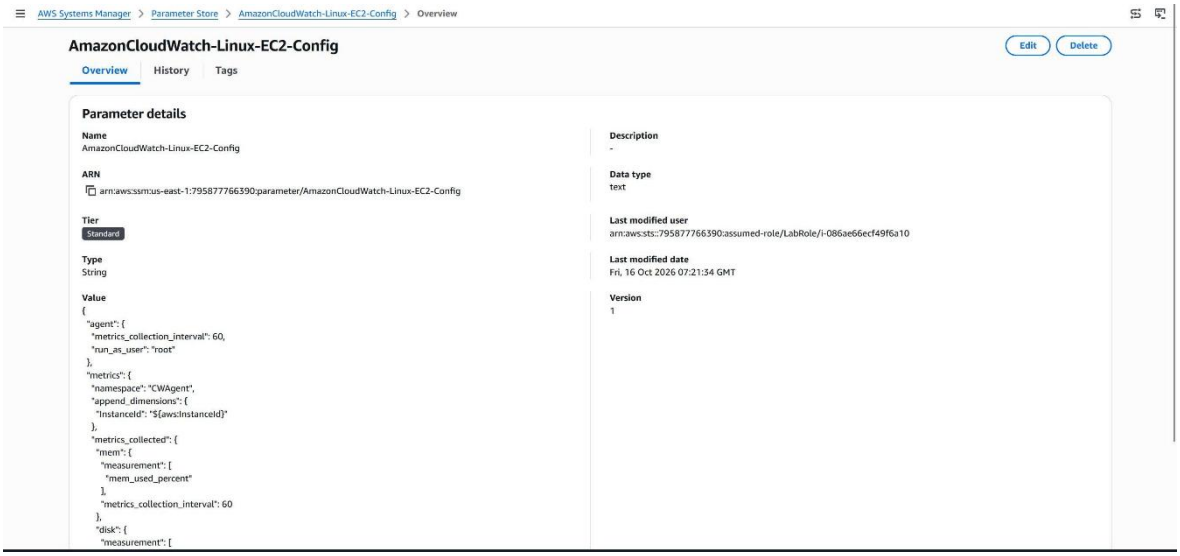
Kết quả triển khai cho thấy CloudWatch Agent có thể thu thập dữ liệu tài nguyên và nhật ký từ EC2, bổ sung các thông tin mà bộ Metrics mặc định chưa cung cấp đầy đủ. Đây là cơ sở để nhóm tiếp tục xây dựng Metric Filter, Alarm và Dashboard trong các bước tiếp theo.




*Hình 6 Tạo Agent trong EC2*




*Hình 7 Metric của CloudWatch Agent*




*Hình 8 Agent trong Parameter Store*

## 3.2. Alarm, SNS, Dashboard, Metric Filter

Sau khi Agent đã đưa nhật ký và chỉ số lên CloudWatch, phần này trình bày cách nhóm biến dữ liệu đó thành cảnh báo và giao diện theo dõi, tạo Metric Filter để đếm lỗi HTTP 5xx, thiết lập SNS để gửi email, tạo Alarm theo dõi chỉ số lỗi và xây dựng Dashboard tổng hợp.

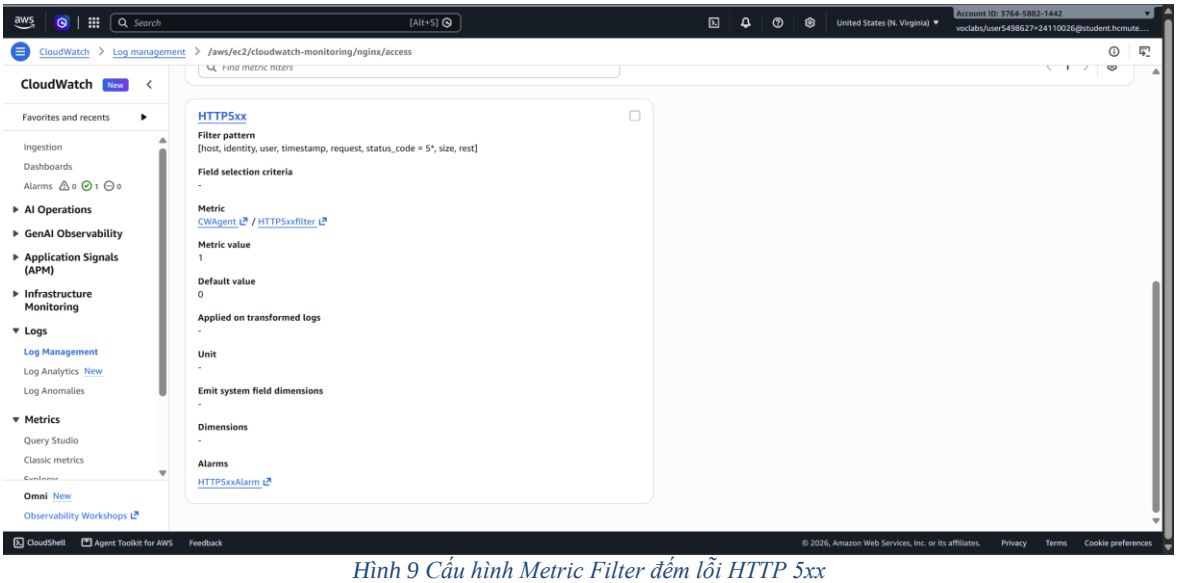
### 3.2.1 Tạo Metric Filter đếm lỗi HTTP 5xx

Access Logs của Nginx ghi lại mã trạng thái HTTP của từng yêu cầu, nhưng bản thân log không thể dùng trực tiếp để đặt Alarm. Vì vậy nhóm tạo một Metric Filter trên Log Group /aws/ec2/cloudwatch-monitoring/nginx/access. Filter sẽ quét từng dòng log, nếu mã trạng thái nằm trong khoảng 500–599 thì cộng thêm 1 vào một chỉ số tùy chỉnh trong CloudWatch Metrics. Nhờ đó, số lượng lỗi máy chủ theo thời gian được chuyển thành dữ liệu có thể vẽ biểu đồ và đặt ngưỡng cảnh báo.

| Thông số | Giá trị |
|---|---|
| Log Group | /aws/ec2/cloudwatch-monitoring/nginx/access |
| Filter pattern | [host, identity, user, timestamp, request, status_code = 5*, size, rest] |
| Tên Metric Filter | HTTP5xx |
| Metric namespace / Metric name | CWAgent / HTTP5xxfilter |

| Thông số | Giá trị |
|---|---|
| Metric value | 1 (mỗi dòng log lỗi 5xx được tính một lần) |
| Default value | 0 |

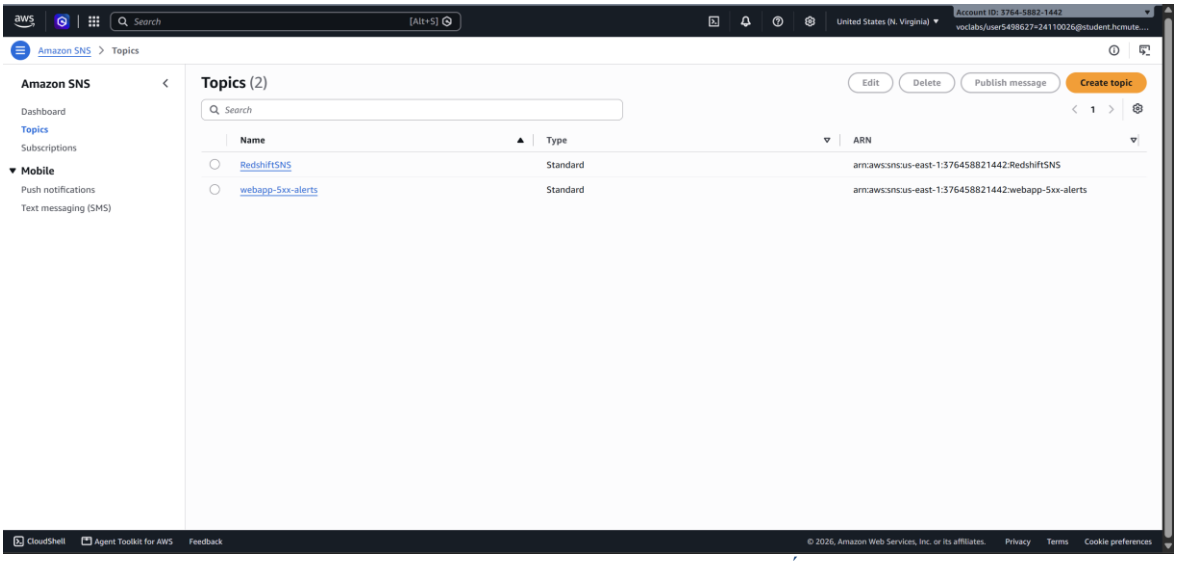
Lưu ý, Metric Filter chỉ áp dụng cho các dòng log được gửi đến sau khi filter được tạo, nên nhóm phải tạo thêm lưu lượng truy cập mới để kiểm tra filter hoạt động.



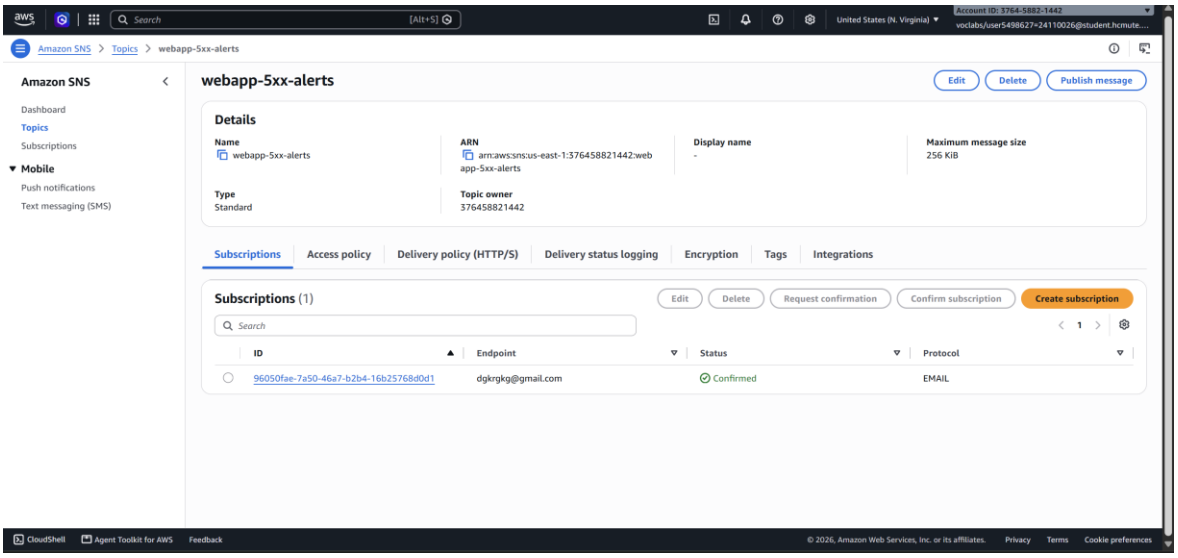

*Hình 9 Cấu hình Metric Filter đếm lỗi HTTP 5xx*

### 3.2.2 Tạo SNS Topic và đăng ký email nhận cảnh báo

Để cảnh báo được gửi đến người quản trị, nhóm sử dụng Amazon SNS. Nhóm tạo một Topic loại Standard tên webapp-5xx-alerts, sau đó tạo Subscription với giao thức Email và địa chỉ email của nhóm. Ngay sau khi tạo, Subscription ở trạng thái Pending confirmation. SNS gửi một email chứa liên kết xác nhận; nhóm mở email và chọn Confirm subscription. Khi trạng thái chuyển sang Confirmed, Topic mới có thể gửi thông báo đến email này. Nếu bỏ qua bước xác nhận, email cảnh báo sẽ không được gửi dù Alarm đã kích hoạt.




*Hình 10 SNS Topic nhận cảnh báo của hệ thống*




*Hình 11 SNS Subscription ở trạng thái Confirmed*

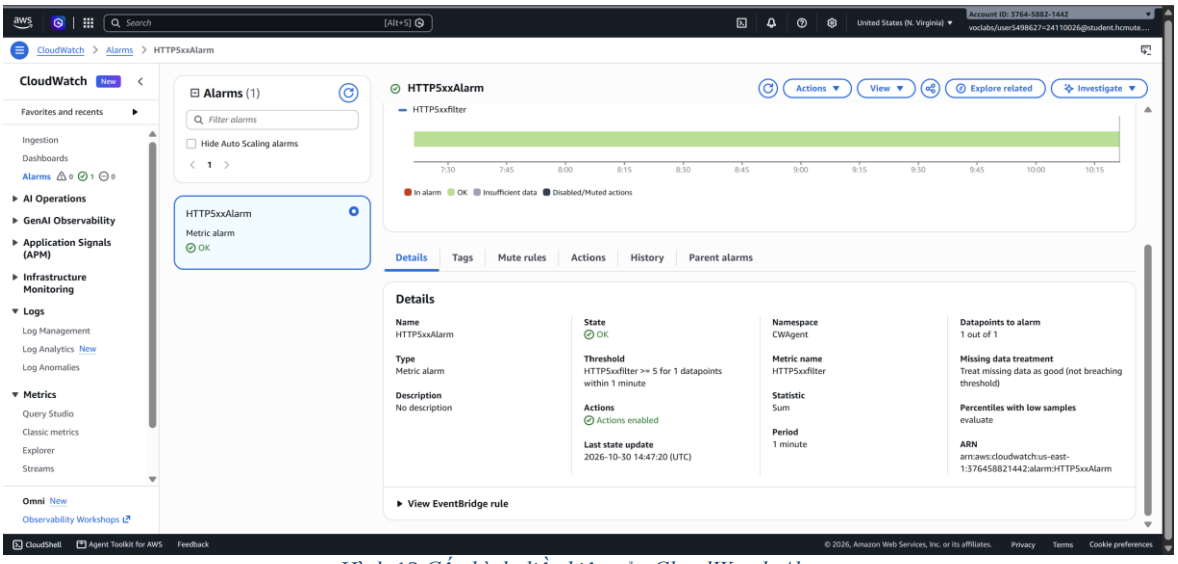
### 3.2.3 Tạo CloudWatch Alarm cho lỗi HTTP 5xx

Nhóm tạo một CloudWatch Alarm dựa trên chỉ số lỗi 5xx do Metric Filter tạo ra. Alarm cộng dồn số lỗi trong mỗi khoảng thời gian đánh giá và so sánh với ngưỡng đã đặt. Khi số lỗi vượt ngưỡng, Alarm chuyển sang trạng thái In alarm và thực hiện hành động đã cấu hình là gửi thông báo đến SNS Topic ở trên. Khi lỗi không còn, Alarm tự trở về trạng thái OK.

| Thông số | Giá trị |
|---|---|
| Chỉ số theo dõi | HTTP5xxfilter (namespace CWAgent) |
| Statistic / Period | Sum / 1 phút |
| Điều kiện (Threshold) | Static, Greater/Equal (>=) 5 |

| Thông số | Giá trị |
|---|---|
| Datapoints to alarm | 1 trên 1 |
| Treat missing data | Treat missing data as good (not breaching) |
| Hành động khi In alarm | Gửi thông báo đến SNS Topic webapp-5xx-alerts |
| Tên Alarm | HTTP5xxAlarm |

Việc đặt cách xử lý dữ liệu bị thiếu rất quan trọng: khi không có lỗi nào xảy ra, Metric Filter không sinh điểm dữ liệu, nếu không cấu hình phù hợp Alarm sẽ ở trạng thái Insufficient data thay vì OK.




*Hình 12 Cấu hình điều kiện của CloudWatch Alarm*

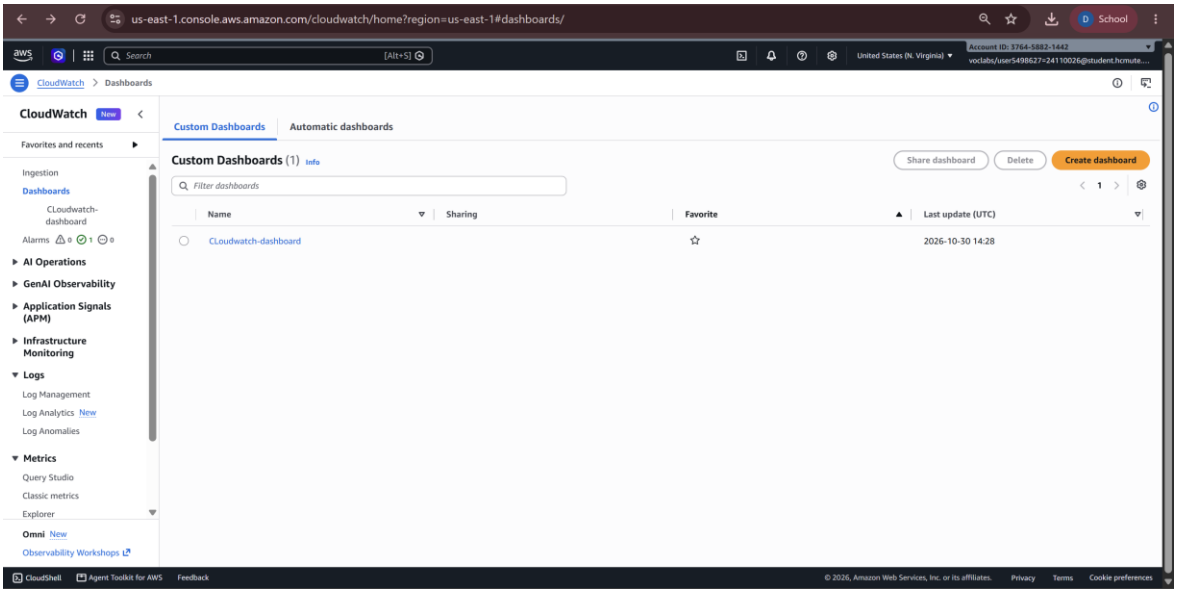
### 3.2.4 Xây dựng CloudWatch Dashboard

Cuối cùng, nhóm xây dựng một CloudWatch Dashboard để quan sát toàn bộ tình trạng máy chủ trên một giao diện. Dashboard gồm các widget sau:

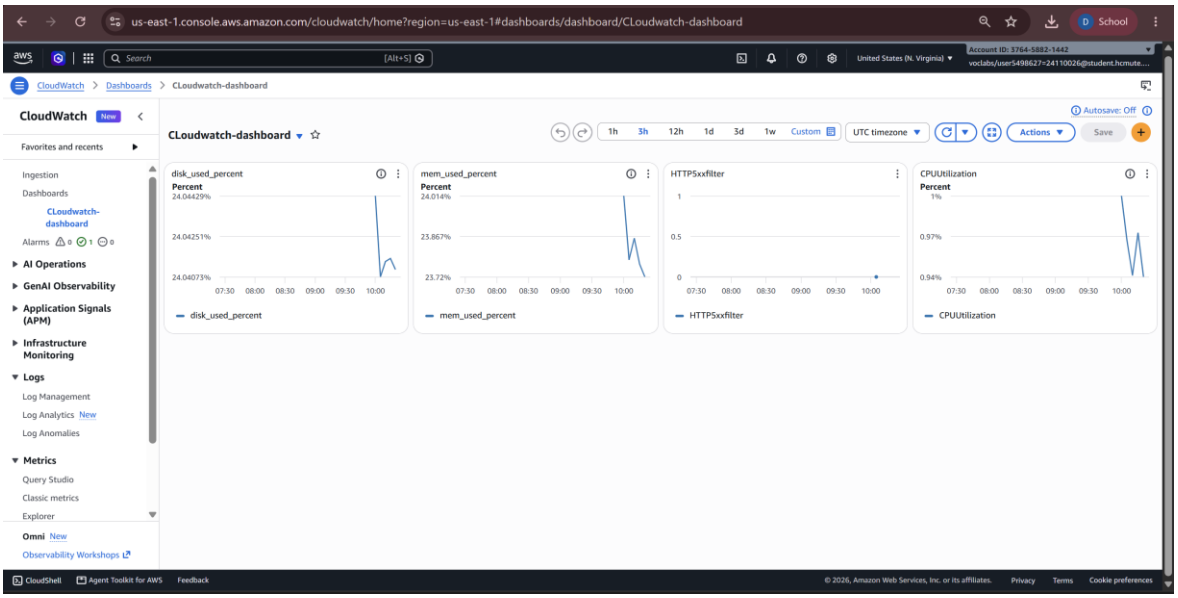
- Biểu đồ CPUUtilization của EC2 (namespace AWS/EC2).
- Biểu đồ mem_used_percent là mức sử dụng RAM.
- Biểu đồ disk_used_percent là mức sử dụng ổ đĩa tại phân vùng / (namespace CWAgent).
- Biểu đồ số lỗi HTTP 5xx do Metric Filter tạo ra.

Nhóm tạo Dashboard trước, sau đó lần lượt thêm từng widget bằng cách chọn chỉ số tương ứng và sắp xếp lại bố cục cho dễ theo dõi. Dashboard giúp người quản trị nhìn thấy ngay khi CPU, RAM, Disk hoặc số lỗi 5xx

tăng bất thường mà không phải mở từng dịch vụ riêng lẻ.




*Hình 13 Tạo CloudWatch Dashboard*




*Hình 14 Dashboard giám sát sau khi hoàn thiện*

## 4. Bảo mật

Trong quá trình triển khai hệ thống giám sát trên nền tảng AWS, bảo mật là một yếu tố quan trọng nhằm đảm bảo các tài nguyên được truy cập đúng đối tượng, hạn chế những kết nối không cần thiết và bảo vệ dữ liệu trong quá trình vận hành. Nhóm tập trung thực hiện hai nội dung chính là quản lý quyền truy cập bằng AWS Identity and Access Management và kiểm soát kết nối mạng thông qua Amazon VPC và Security Group.

Nhóm sử dụng IAM Role có sẵn với tên LabRole trong môi trường AWS Academy Learner Lab để cấp quyền cho máy chủ EC2 truy cập và sử dụng các dịch vụ AWS cần thiết. Việc sử dụng IAM Role giúp EC2 có thể thực hiện các thao tác liên quan đến hệ thống giám sát mà không cần lưu trữ Access Key hoặc Secret Key trực tiếp trên máy chủ.

Trong phạm vi dự án, EC2 cần có quyền gửi các chỉ số tài nguyên như RAM, Disk lên Amazon CloudWatch Metrics, đồng thời gửi nhật ký hoạt động của Nginx lên CloudWatch Logs. Ngoài ra, nhóm sử dụng AWS Systems Manager Parameter Store để lưu trữ cấu hình CloudWatch Agent, hỗ trợ quản lý và tái sử dụng cấu hình khi cần thiết.

Để bảo vệ máy chủ và kiểm soát lưu lượng truy cập, nhóm triển khai hệ thống trong môi trường mạng riêng ảo Amazon VPC. VPC được cấu hình với dải địa chỉ IPv4 CIDR là 10.0.0.0/16, tạo không gian địa chỉ riêng cho các tài nguyên thuộc dự án. Việc sử dụng VPC giúp nhóm quản lý môi trường mạng và kiểm soát kết nối giữa máy chủ EC2 với các dịch vụ liên quan.

Bên cạnh VPC, nhóm sử dụng Security Group để kiểm soát lưu lượng mạng đi vào máy chủ EC2. Security Group hoạt động như một tường lửa ảo, cho phép xác định những cổng kết nối và nguồn truy cập được phép giao tiếp với máy chủ. Thông qua việc kết hợp IAM Role, VPC, Security Group nhóm đã xây dựng các lớp bảo mật cơ bản cho hệ thống giám sát, góp phần kiểm soát quyền truy cập và giảm thiểu những rủi ro bảo mật trong quá trình triển khai.

Nhìn chung, hệ thống đã áp dụng các cơ chế bảo mật cơ bản của AWS để quản lý quyền truy cập và kiểm soát kết nối mạng. IAM Role giúp EC2 sử dụng các dịch vụ AWS mà không cần lưu trữ khóa truy cập trực tiếp. VPC cung cấp môi trường mạng riêng, trong khi Security Group kiểm soát các cổng kết nối cần thiết cho hoạt động quản trị và truy cập website. Tuy nhiên, hệ thống hiện mới tập trung vào các biện pháp bảo mật cơ bản phục vụ môi trường học tập và thử nghiệm. Trong tương lai, nhóm có thể tiếp tục cải thiện

bằng cách áp dụng nguyên tắc cấp quyền tối thiểu, giới hạn chặt chẽ địa chỉ IP được phép SSH, sử dụng HTTPS với chứng chỉ TLS hợp lệ và tăng cường theo dõi các hoạt động truy cập thông qua AWS CloudTrail.

## 5. Kiểm thử

### 5.1. Mục tiêu và môi trường kiểm thử

Việc kiểm thử nhằm xác nhận toàn bộ chuỗi giám sát hoạt động đúng như thiết kế: từ lúc sự cố xảy ra trên EC2, dữ liệu được ghi vào CloudWatch, Alarm chuyển trạng thái OK → In alarm → OK, email cảnh báo được gửi cho đến khi Dashboard phản ánh được sự cố. Nhóm thực hiện năm kịch bản: lỗi HTTP 5xx, lỗi HTTP 4xx, RAM cao, đầy đĩa và CPU cao.

Môi trường kiểm thử là máy chủ EC2 cloudwatch-monitoring-web (t3.micro, us-east-1a) trong VPC monitoring-vpc, dùng LabRole, chạy Nginx và CloudWatch Agent. Agent gửi nhật ký Nginx lên hai Log Group access và error, đồng thời gửi các chỉ số mem_used_percent và disk_used_percent lên namespace CWAgent; chỉ số CPUUtilization do EC2 cung cấp sẵn. Các lệnh gây lỗi được chạy trên EC2 thông qua AWS Systems Manager Session Manager. Mọi mốc thời gian trong phần này tính theo múi giờ UTC, cùng múi giờ hiển thị trên CloudWatch.

### 5.2. Các kịch bản kiểm thử

Với mỗi kịch bản, nhóm tạo một Alarm riêng, chọn Statistic và Period phù hợp, đặt Datapoints to alarm là 1 trên 1 và Treat missing data là coi như không vi phạm. Ngưỡng được chọn cao hơn mức hoạt động bình thường của máy chủ (RAM khoảng 26%, Disk khoảng 24%, CPU khoảng 5%) để tránh cảnh báo giả, nhưng đủ thấp để có thể tạo ra bằng các lệnh mô phỏng an toàn.

| STT | Alarm và điều kiện | Cách gây lỗi | Kết quả |
|---|---|---|---|
| 1 | HTTP5xxAlarm: HTTP5xxfilter, Sum / 1 phút, >= 5 | Cấu hình Nginx test-500.conf, dùng curl gọi đường dẫn /test-500 | Metric Filter đếm lỗi, Alarm In alarm, SNS gửi email |

| STT | Alarm và điều kiện | Cách gây lỗi | Kết quả |
|---|---|---|---|
| 2 | HTTP4xxAlarm: HTTP4xxfilter, Sum / 1 phút, > 5 | Dùng curl gọi 30 đường dẫn không tồn tại | Metric Filter đếm lỗi 404, Alarm In alarm |
| 3 | HighMemoryAlarm: mem_used_percent, Average / 1 phút, >= 60 | Chạy Python cấp phát 400 MB RAM | RAM vượt 60%, Alarm In alarm |
| 4 | HighDiskAlarm: disk_used_percent, Average / 1 phút, >= 50 | Tạo file 3 GB bằng fallocate | Disk vượt 50%, Alarm In alarm |
| 5 | HighCPUAlarm: CPUUtilization, Average / 5 phút, >= 40 | Chạy 2 tiến trình yes > /dev/null | CPU vượt 40%, Alarm In alarm |

## 5.3. Kết quả kiểm thử chi tiết

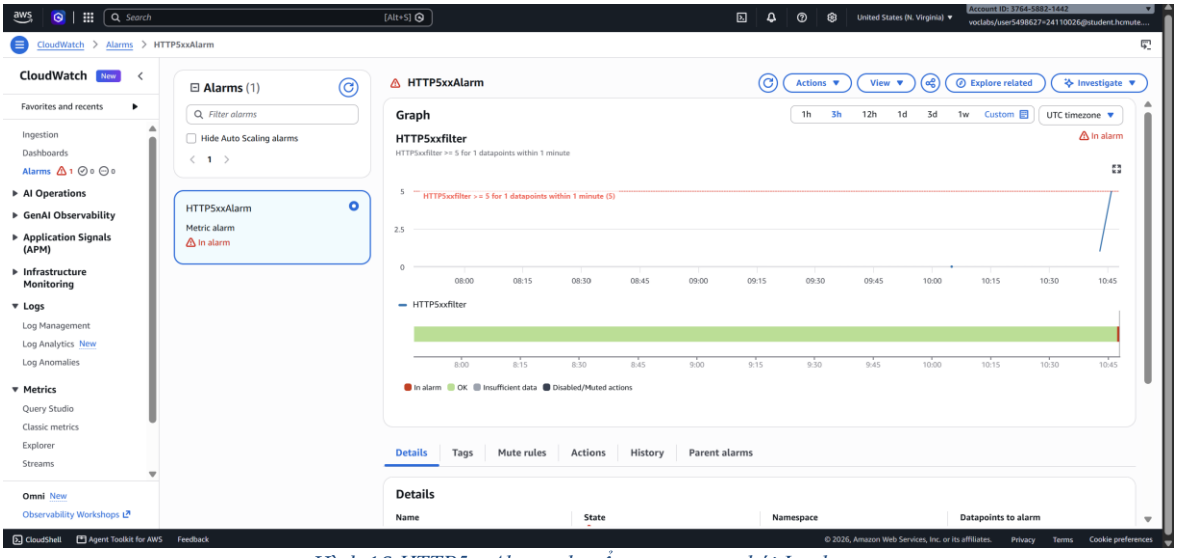
### 5.3.1 Kiểm thử lỗi HTTP 5xx

Nhóm thêm cấu hình Nginx test-500.conf để một đường dẫn kiểm thử (/test-500) luôn trả về mã lỗi 500, sau đó dùng curl gọi đường dẫn này nhiều lần. Các dòng log mã 500 được Agent đưa lên Log Group access, Metric Filter đếm thành chỉ số HTTP5xxfilter và tổng số lỗi trong một phút vượt ngưỡng 5. HTTP5xxAlarm chuyển sang In alarm, SNS gửi email cảnh báo đến địa chỉ đã đăng ký và biểu đồ lỗi 5xx trên Dashboard tăng đột biến. Khi ngừng gửi yêu cầu, Alarm tự trở về OK.

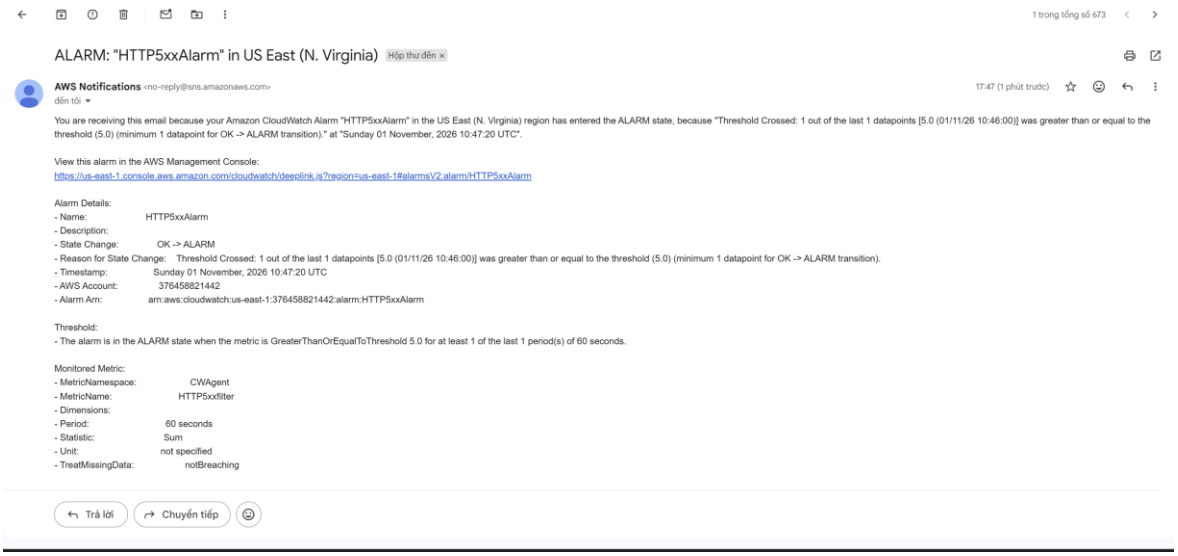
```
for i in {1..5}; do curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:8080/test-500; done
```

```
sh-5.2$ for i in {1..5}; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/test-500; done
500
500
500
500
500
sh-5.2$
```

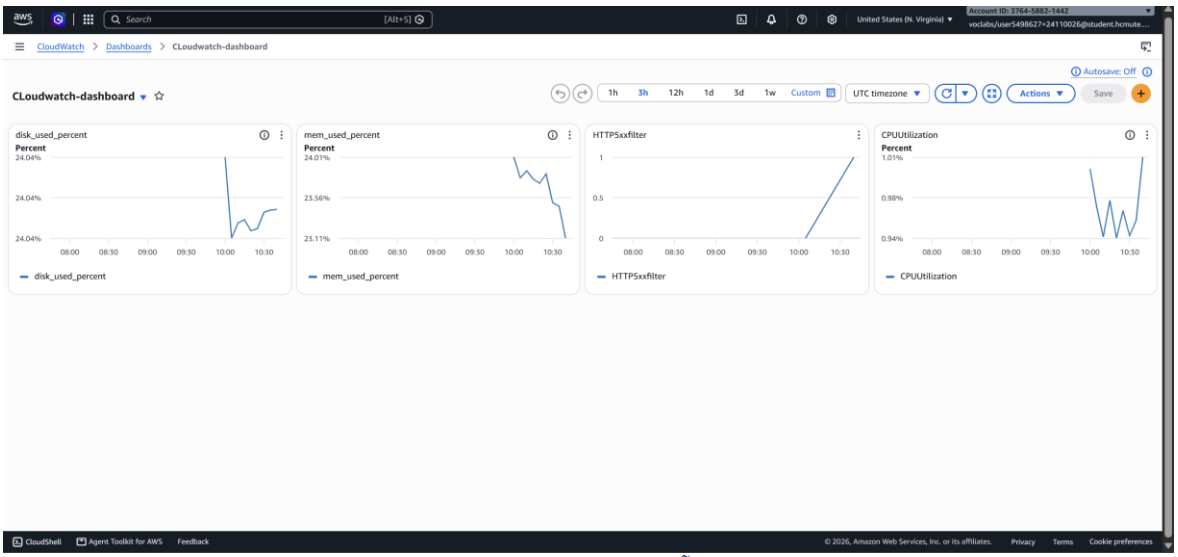
*Hình 15 Gọi đường dẫn /test-500 để tạo lỗi HTTP 500*




*Hình 16 HTTP5xxAlarm chuyển sang trạng thái In alarm*




*Hình 17 Email cảnh báo nhận được từ Amazon SNS*




*Hình 18 Dashboard ghi nhận lỗi HTTP 5xx*

### 5.3.2 Kiểm thử lỗi HTTP 404

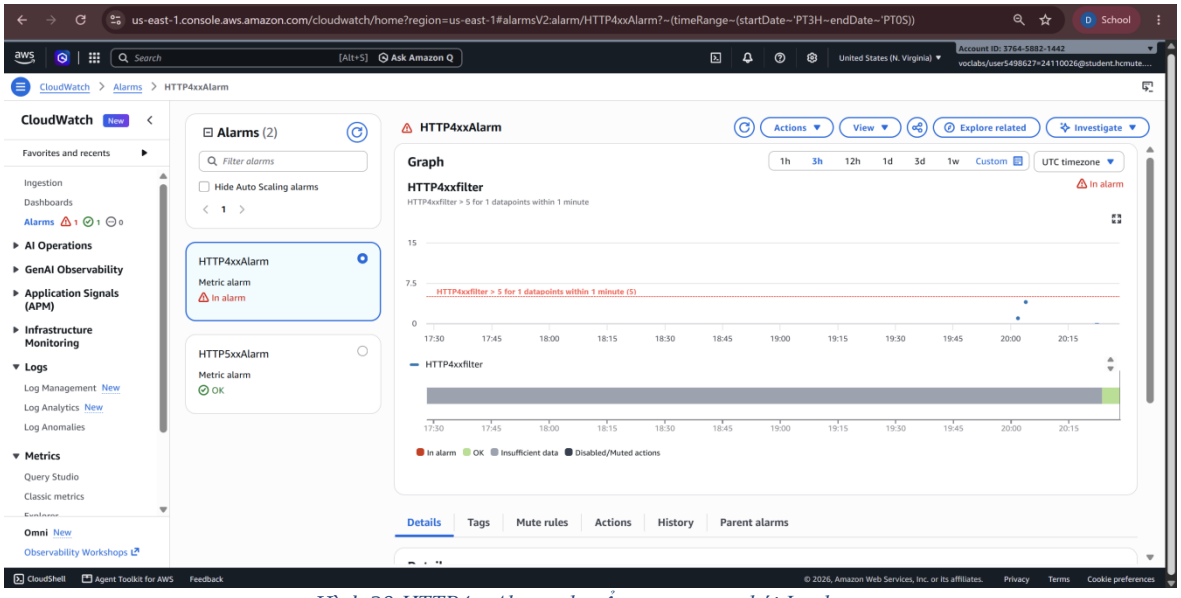
Nhóm tạo thêm Metric Filter trên Log Group access với pattern là chuỗi " 404 ". Ban đầu nhóm thử pattern dạng trường [ip, id, ...] nhưng CloudWatch báo lỗi ký tự không hợp lệ, nên chuyển sang pattern chuỗi; chức năng Test pattern cho kết quả 7 dòng khớp trong 39 dòng log. Metric được ghi vào namespace CWAgent với tên HTTP4xxfilter, giá trị 1 và Default value là 0. HTTP4xxAlarm dùng Sum theo 1 phút với ngưỡng lớn hơn 5.

Để gây lỗi, nhóm dùng vòng lặp curl gửi 30 yêu cầu đến các đường dẫn không tồn tại. Alarm chuyển sang In alarm; thanh trạng thái cho thấy chuỗi Insufficient data → OK → In alarm. Trạng thái Insufficient data ban đầu xuất hiện vì Metric Filter chỉ phát dữ liệu khi có log mới, và được khắc phục bằng cách gửi một vài yêu cầu bình thường. Trong log còn ghi nhận cả các yêu cầu 404 từ bot trên Internet quét địa chỉ IP công khai, chứng tỏ Alarm hoạt động được với lưu lượng thật. Kịch bản này không cần dọn dẹp vì Alarm tự trở về OK khi không còn lỗi mới.

```
for i in {1..30}; do curl -s -o /dev/null -w "%{http_code} "
http://localhost:8080/loi-404-$i; done
```




*Hình 19 Chạy vòng lặp curl tạo lỗi HTTP 404*




*Hình 20 HTTP4xxAlarm chuyển sang trạng thái In alarm*

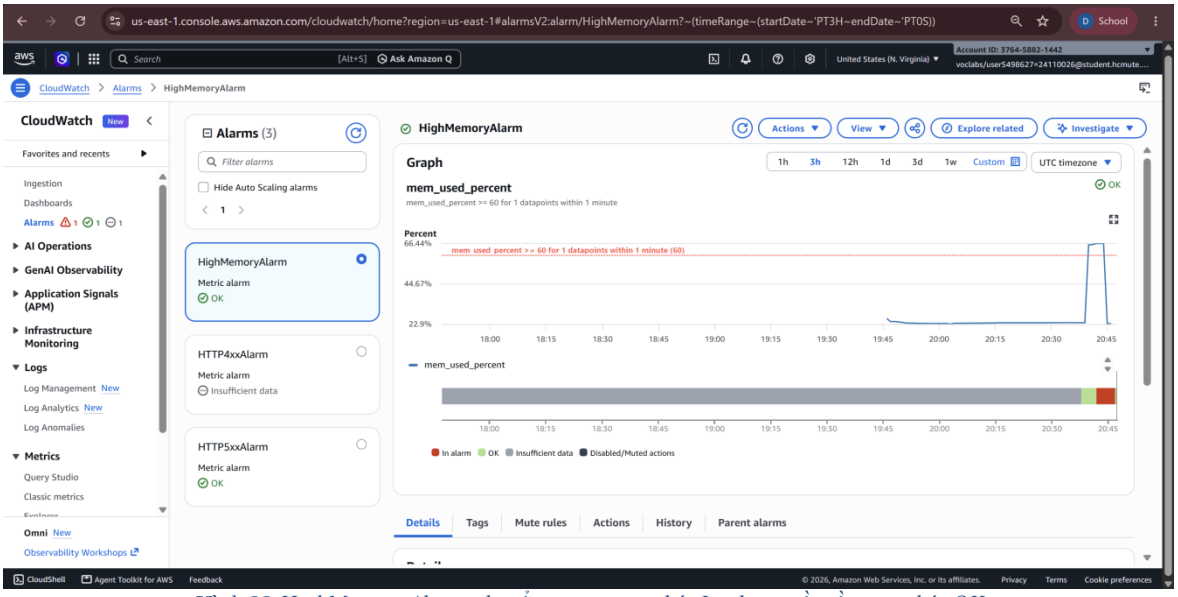
### 5.3.3 Kiểm thử RAM sử dụng cao

HighMemoryAlarm theo dõi chỉ số mem_used_percent bằng Average trong 1 phút với ngưỡng 60%. Ở trạng thái bình thường RAM chỉ khoảng 26%. Nhóm chạy một tiến trình Python cấp phát 400 MB bộ nhớ và giữ trong 10 phút. Mức sử dụng RAM tăng lên khoảng 65,67%, vượt ngưỡng và Alarm chuyển sang In alarm. Nhóm chọn mức 400 MB thay vì 650 MB để tránh làm máy chủ t3.micro bị treo. Sau kiểm thử, nhóm dừng tiến trình bằng pkill, kiểm tra lại bằng free -m và chờ Alarm trở về OK.

```
python3 -c "x='a'*(400*1024*1024); import time; time.sleep(600)" &
pkill python3
```

```
sh-5.2$    free -m
               total        used        free      shared  buff/cache   available
Mem:             913         216         277           0         419         565
Swap:              0           0           0
sh-5.2$    python3 -c "x='a'*(400*1024*1024); import time; time.sleep(600)" &
   sleep 5
   free -m
[1] 3677
               total        used        free      shared  buff/cache   available
Mem:             913         600          72           0         239         180
Swap:              0           0           0
sh-5.2$ pkill python3
free -m
               total        used        free      shared  buff/cache   available
Mem:             913         585          80           0         247         195
Swap:              0           0           0
sh-5.2$
```

*Hình 21 Chạy tiến trình chiếm RAM trên EC2*




*Hình 22 HighMemoryAlarm chuyển sang trạng thái In alarm rồi về trạng thái OK*

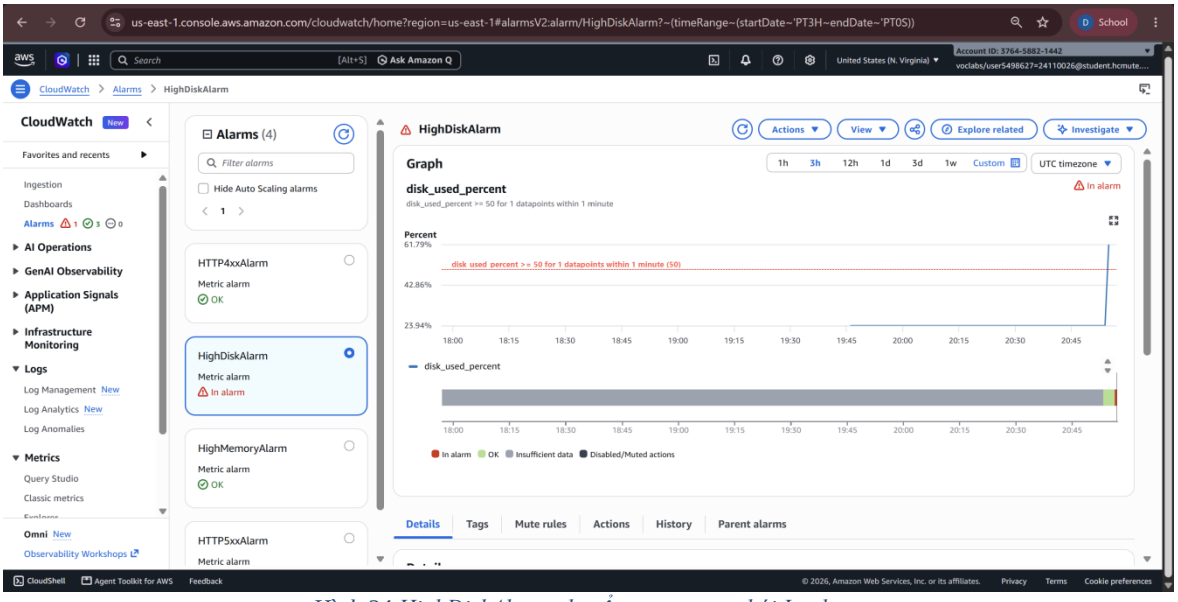
### 5.3.4 Kiểm thử ổ đĩa đầy

HighDiskAlarm theo dõi chỉ số disk_used_percent của phân vùng / bằng Average trong 1 phút với ngưỡng 50%, trong khi mức sử dụng bình thường khoảng 24%. Nhóm tạo một file 3 GB trong thư mục /var vì người dùng ssm-user của Session Manager không có quyền ghi vào thư mục home của ec2-user. Mức sử dụng đĩa vượt 50% và Alarm chuyển sang In alarm. Đây là kịch bản bắt buộc phải dọn dẹp: nhóm xóa file đã tạo, kiểm tra lại bằng df -h và chờ Alarm trở về OK.

```
sudo fallocate -l 3G /var/bigfile.img
sudo rm -f /var/bigfile.img
```

```
sh-5.2$    df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  1.9G  6.1G  24% /
[1]+  Terminated              python3 -c "x='a'*(400*1024*1024); import time; time.sleep(600)"
sh-5.2$ df -h /
sudo fallocate -l 3G /var/bigfile.img
df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  1.9G  6.1G  24% /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  4.9G  3.1G  62% /
sh-5.2$ sudo rm -f /var/bigfile.img /var/bigfile2.img
df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  1.9G  6.1G  24% /
sh-5.2$
```

*Hình 23 Tạo file 3 GB để làm đầy ổ đĩa*




*Hình 24 HighDiskAlarm chuyển sang trạng thái In alarm*

### 5.3.5 Kiểm thử CPU sử dụng cao

HighCPUAlarm theo dõi chỉ số CPUUtilization của EC2 bằng Average trong 5 phút với ngưỡng 40%, trong khi CPU bình thường chỉ khoảng 5%. Máy chủ t3.micro có 2 vCPU nên nhóm chạy hai tiến trình yes để làm CPU hoạt động hết công suất. Do độ phân giải của chỉ số là 5 phút, Alarm cần khoảng 5 đến 10 phút mới chuyển sang In alarm; tương tự, sau khi dừng tiến trình bằng pkill, cũng cần 5 đến 10 phút để Alarm trở về OK.

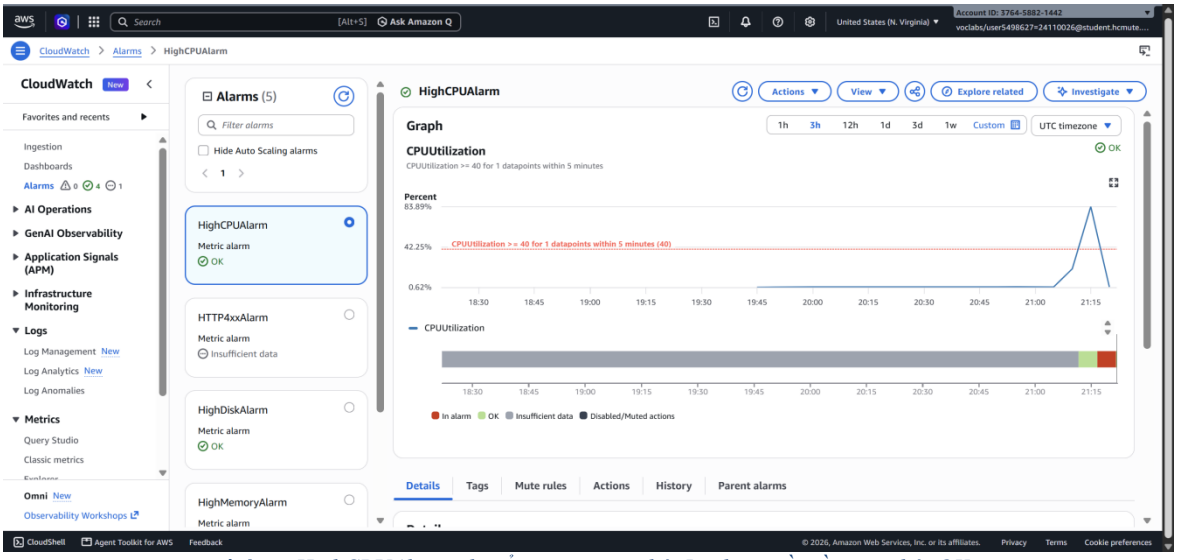
```
yes > /dev/null &
yes > /dev/null &
pkill yes
```

```
sh-5.2$    yes > /dev/null &
   yes > /dev/null &
   sleep 3
   top -bn1 | head -12
[1] 4815
[2] 4816
top - 21:13:28 up  1:27,  0 users,  load average: 0.00, 0.00, 0.00
Tasks: 110 total,   3 running, 107 sleeping,   0 stopped,   0 zombie
%Cpu(s): 61.8 us, 38.2 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    913.1 total,    430.3 free,    226.3 used,    256.5 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    555.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4816 ssm-user  20   0  221388   1760   1640 R 100.0   0.2   0:03.16 yes
   4815 ssm-user  20   0  221388   1836   1716 R 100.0   0.2   0:03.15 yes
      1 root      20   0  173936  17796  11184 S   0.0   1.9   0:01.25 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
sh-5.2$ pkill yes
top -bn1 | head -12
[2]+  Terminated              yes > /dev/null
[1]+  Terminated              yes > /dev/null
top - 21:18:33 up  1:32,  0 users,  load average: 2.06, 1.31, 0.58
Tasks: 107 total,   1 running, 106 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 96.8 id,  0.0 wa,  0.0 hi,  3.2 si,  0.0 st
MiB Mem :    913.1 total,    430.3 free,    226.2 used,    256.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    555.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  173936  17796  11184 S   0.0   1.9   0:01.26 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
sh-5.2$
```

*Hình 25 Chạy hai tiến trình yes để tăng tải CPU*




*Hình 26 HighCPUAlarm chuyển sang trạng thái In alarm rồi về trạng thái OK*

## 5.4. Đánh giá kết quả

Cả năm kịch bản đều cho kết quả đúng thiết kế: Alarm chuyển từ OK sang In alarm khi vượt ngưỡng và tự trở về OK sau khi sự cố được xử lý; riêng lỗi 5xx còn kèm email cảnh báo qua SNS. Một số nhận xét rút ra từ quá trình kiểm thử:

- Độ trễ phát hiện: các Alarm dựa trên chu kỳ 1 phút phát hiện sau vài phút,

trong khi CPUUtilization của EC2 có độ phân giải 5 phút nên chậm nhất. Biểu đồ trên trang Alarm cũng cập nhật chậm hơn trạng thái Alarm vài phút, cần làm mới trang để thấy điểm vượt ngưỡng.

- Alarm dựa trên log: chuỗi xử lý gồm Nginx ghi log, CloudWatch Agent gửi log, Metric Filter tạo chỉ số rồi mới đến Alarm. Metric Filter chỉ đếm các dòng log phát sinh sau khi được tạo, và chỉ có dữ liệu khi có log mới nên Alarm có thể ở trạng thái Insufficient data; cấu hình Treat missing data as good giúp tránh cảnh báo nhầm.
- Chọn ngưỡng: ngưỡng cần cao hơn mức hoạt động bình thường để tránh cảnh báo giả nhưng vẫn đủ thấp để phát hiện sự cố sớm. Với RAM và Disk, nhóm chọn mức mô phỏng vừa đủ vượt ngưỡng (400 MB RAM, file 3 GB) để không làm ảnh hưởng đến máy chủ.

Hạn chế: kiểm thử mới thực hiện trên một máy chủ EC2 và các sự cố đều được mô phỏng bằng lệnh, chưa phản ánh đầy đủ lưu lượng và sự cố của môi trường thực tế.

**6. Chi phí**

Trong quá trình triển khai và vận hành hệ thống CloudWatch Monitoring, nhóm sử dụng nhiều dịch vụ thuộc nền tảng Amazon Web Services như Amazon EC2, Amazon VPC, Amazon CloudWatch và Amazon S3. Để theo dõi và đánh giá mức chi phí phát sinh, nhóm sử dụng công cụ AWS Cost Explorer trong AWS Billing and Cost Management nhằm thống kê chi phí theo từng dịch vụ trong tháng 10.

Theo dữ liệu ghi nhận trên AWS Cost Explorer, tổng chi phí sử dụng các dịch vụ AWS của cả hai thành viên trong tháng 10 là 0,56 USD. Trong đó, dịch vụ EC2-Other phát sinh chi phí cao nhất với 0,32 USD, tiếp theo là Amazon Elastic Compute Cloud - Compute với 0,07 USD. Hai dịch vụ Amazon Virtual Private Cloud và Amazon CloudWatch cùng phát sinh chi phí 0,05 USD.

Qua kết quả thống kê, có thể nhận thấy phần lớn chi phí tập trung vào các tài nguyên liên quan đến EC2, bao gồm chi phí EC2-Other và chi phí sử

dụng máy chủ tính toán. CloudWatch phát sinh chi phí ở mức thấp so với tổng chi phí, mặc dù được sử dụng để thu thập Metrics, lưu trữ Logs, thiết lập Alarm và Dashboard phục vụ giám sát hệ thống.

Bên cạnh việc triển khai các chức năng giám sát, nhóm cũng chú trọng đến việc kiểm soát và tối ưu chi phí nhằm hạn chế các khoản phát sinh không cần thiết trong quá trình vận hành hệ thống. Đối với Amazon EC2, nhóm thực hiện dừng máy chủ khi không có nhu cầu sử dụng hoặc khi đã hoàn thành các bài kiểm thử. Việc này giúp giảm chi phí tính toán phát sinh trong thời gian máy chủ không hoạt động. Tuy nhiên, nhóm vẫn lưu ý rằng một số tài nguyên liên quan như EBS Volume và Elastic IP có thể tiếp tục phát sinh chi phí ngay cả khi EC2 đã được dừng.

Đối với Amazon CloudWatch Logs, nhóm thiết lập thời gian lưu trữ dữ liệu là 7 ngày cho các Log Groups chứa Access Logs và Error Logs của Nginx. Việc giới hạn thời gian lưu trữ giúp tránh tích lũy quá nhiều dữ liệu nhật ký trong thời gian dài, từ đó hỗ trợ kiểm soát dung lượng lưu trữ và chi phí.

Ngoài ra, nhóm thiết lập chu kỳ thu thập Metrics của CloudWatch Agent là 60 giây đối với các chỉ số RAM và Disk. Chu kỳ này đáp ứng nhu cầu theo dõi tình trạng máy chủ trong phạm vi dự án mà không cần thu thập dữ liệu với tần suất quá dày. Nhóm cũng sử dụng một CloudWatch Dashboard để tổng hợp các thông tin giám sát quan trọng, bao gồm CPU, RAM, Disk và HTTP 5xx, giúp quản lý dữ liệu tập trung và thuận tiện hơn.

Sau khi hoàn thành quá trình triển khai, kiểm thử và đánh giá hệ thống, nhóm tiến hành rà soát các tài nguyên AWS nhằm loại bỏ những thành phần không còn cần thiết, tránh phát sinh chi phí ngoài dự kiến và giữ cho môi trường triển khai được gọn gàng. Đối với CloudWatch, nhóm kiểm tra lại các Log Groups, Metric Filters, Alarms và Dashboard để xác định những thành phần cần tiếp tục duy trì và những thành phần chỉ phục vụ thử nghiệm. Các thành phần quan trọng như CloudWatch Dashboard, Alarm, SNS Topic và Log Groups được giữ lại trong thời gian cần thiết để phục vụ việc demo,

kiểm tra và đánh giá kết quả dự án.

## 7. Vấn đề gặp phải và bài học rút ra

Vấn đề 1: Không thể truy cập website trên EC2 thông qua Public Ipv4. Trong quá trình triển khai Nginx trên EC2, mặc dù máy chủ đã được khởi chạy nhưng khi truy cập địa chỉ Public IPv4 bằng trình duyệt, nhóm gặp lỗi ERR_CONNECTION_TIMED_OUT. Điều này cho thấy yêu cầu truy cập từ bên ngoài chưa đến được dịch vụ web hoặc chưa nhận được phản hồi. Để khắc phục nhóm tiến hành kiểm tra trạng thái EC2, cấu hình Security Group và khả năng truy cập mạng. Sau đó, nhóm kiểm tra dịch vụ Nginx trên EC2 bằng lệnh systemctl status nginx, xác nhận dịch vụ đã ở trạng thái active. Đồng thời, nhóm kiểm tra lại cấu hình cho phép truy cập HTTP qua cổng 80. Sau khi điều chỉnh và kiểm tra, website đã có thể truy cập thành công thông qua Public IPv4.

Vấn đề 2: Không tìm thấy file cấu hình CloudWatch Agent Khi tiến hành kiểm tra cấu hình CloudWatch Agent, nhóm sử dụng lệnh cat để đọc file /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json nhưng nhận được thông báo No such file or directory. Để khắc phục nhóm xác định file cấu hình chưa được tạo tại đường dẫn trên, sau đó tiến hành tạo file JSON và khai báo các thông số cần thu thập như mức sử dụng RAM mem_used_percent, dung lượng ổ đĩa disk_used_percent với chu kỳ 60 giây. Nhóm tiếp tục khởi động CloudWatch Agent bằng lệnh amazon-cloudwatch-agent-ctl và kiểm tra trạng thái hoạt động của Agent

## 8. Phân công và khai báo sử dụng AI

| | **Nguyễn Tuấn Khang** | **Đào Gia Khiêm** | **AI Hỗ trợ** |
|---|---|---|---|
| **Tuần 1** | Dựng nền tảng VPC, S3 Bucket và cấu hình Tagging<br>Tìm hiểu Amazon S3, IAM LabRole và Systems Manager | Tìm hiểu EC2, VPC, subnet, route table và Internet Gateway<br>Thiết kế sơ đồ kiến trúc hệ thống và luồng dữ liệu giữa EC2, | Gemini 3.8 Flash<br>AI tạo ra: Các lệnh AWS CLI để tạo S3 Bucket và VPC. Đề xuất mô hình kiến trúc sau khi chạy |

| | | | |
|---|---|---|---|
| | Parameter Store<br>Tìm hiểu cách tạo S3 bucket và mục đích lưu trữ tài liệu, template của dự án | CloudWatch, S3, Parameter Store và SNS<br>Kiểm tra Region và quyền LabRole trên AWS Learner Lab | script. Yêu cầu kiểm tra các quyền của LabRole trước khi chạy. Tạo code kiểm tra tài nguyên sau khi tạo.<br>SV đã sửa: Tự chạy toàn bộ lệnh trên AWS Learner Lab. Tự tạo các cài đặt còn thiếu cho các tài nguyên VPC, S3 Bucket và gắn tag. Sau khi tạo hoàn tất chụp ảnh giao diện để đưa vào báo cáo. Điều chỉnh các nội dung do AI gợi ý cho phù hợp với yêu cầu của đề tài. |
| **Tuần 2** | Kiểm tra VPC, subnet, Internet Gateway, route table đã tạo ở tuần 1<br>Tạo EC2 chạy ứng dụng web thử nghiệm, kiểm tra truy cập và trạng thái hoạt động<br>Cài đặt CloudWatch Agent trên EC2 và cấu hình Agent thu thập các chỉ số bổ sung như RAM, dung lượng đĩa và mức sử dụng đĩa | Tạo CloudWatch Log Group để nhận log ứng dụng web hoặc web server từ EC2<br>Thiết lập thời gian lưu log (retention 7 ngày)<br>Tạo Metric Filter để đếm lỗi HTTP 5xx trong log<br>Tạo SNS Topic và đăng ký email nhận cảnh báo, xác nhận đăng ký email<br>Tạo thử một CloudWatch Alarm | Claude Sonnet 5.5<br>AI tạo ra: Các code tạo dữ liệu lỗi. Đề xuất cài đặt alarm<br>Kiểm thử và chỉnh sửa lại cho đúng đề tài trên AWS Learner Lab |

| | | | |
|---|---|---|---|
| | Lưu cấu hình Agent trong Systems Manager Parameter Store<br>Khởi động Agent, kiểm tra dữ liệu được gửi lên CloudWatch | dựa trên metric lỗi 5xx và cấu hình gửi thông báo qua SNS<br>Tạo dữ liệu lỗi thử nghiệm để kiểm tra Metric Filter và Alarm | |
| **Tuần 3** | Kiểm tra EC2, Nginx và cấu hình CloudWatch Agent<br>Theo dõi metrics CPU, RAM và dung lượng đĩa<br>Cấu hình thu thập access log, error log và thời gian lưu trữ log<br>Kiểm tra Security Group, IAM Role và các thiết lập bảo mật cơ bản | Cấu hình Metric Filter đếm lỗi HTTP 5xx<br>Tạo CloudWatch Alarm và tích hợp Amazon SNS<br>Xây dựng Dashboard tập trung các metrics và lỗi HTTP 5xx<br>Kiểm thử điều kiện vượt ngưỡng và xác minh thông báo cảnh báo | Không có |
| **Tuần 4** | Hoàn thiện tài liệu hướng dẫn triển khai, báo cáo tổng kết<br>Đánh giá chi phí, bảo mật và dọn dẹp tài nguyên | Kiểm thử Alarm với các tình huống lỗi khác nhau<br>Hoàn thiện sơ đồ kiến trúc, báo cáo tổng kết | Không có |